Mini Project 2 — Unsupervised Learning

Consumer Behaviour & Shopping Habits Analysis

Student ID: 2025AIML060
Dataset: Shopping Trends and Customer Behaviour Dataset (Kaggle)

This project explores customer shopping patterns using unsupervised learning techniques — specifically clustering algorithms — alongside descriptive analysis to uncover meaningful insights from the dataset.

Part 1 — Data Preprocessing

1.1 Importing Libraries

In [1]:
```
# Standard libraries
import numpy as np
import pandas as pd
import warnings
warnings.filterwarnings('ignore')

# Visualization
import matplotlib.pyplot as plt
import seaborn as sns
plt.rcParams['figure.figsize'] = (10, 6)
plt.rcParams['axes.grid'] = True

# Preprocessing & Metrics
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.decomposition import PCA
from sklearn.metrics import silhouette_score

# Clustering
from sklearn.cluster import KMeans, AgglomerativeClustering, DBSCAN
from scipy.cluster.hierarchy import dendrogram, linkage

print("All libraries loaded successfully.")
```

```
All libraries loaded successfully.
```

1.2 Loading the Dataset

In [2]:
```
# Loading dataset from local CSV file
df_raw = pd.read_csv('Shopping Trends And Customer Behaviour Dataset.csv')

print(f"Dataset shape: {df_raw.shape}")
print(f"\nColumn names:\n{list(df_raw.columns)}")
df_raw.head(5)
```

```
Dataset shape: (3900, 18)

Column names:
['Unnamed: 0', 'Customer ID', 'Age', 'Gender', 'Item Purchased', 'Category', 'Purchase Amount (USD)', 'Location', 'Color', 'Season', 'Review Rating', 'Subscription Status', 'Shipping Type', 'Discount Applied', 'Promo Code Used', 'Previous Purchases', 'Payment Method', 'F
requency of Purchases']
```

Out[2]:

	Unnamed: 0	Customer ID	Age	Gender	Item Purchased	Category	Purchase Amount (USD)	Location	Color	Season	Review Rating	Subscription Status	Shipping Type	Discount Applied	Promo Code Used	Previous Purchases	Payment Method	Frequency of Purchases
0	0	1	55	Male	Blouse	Clothing	53	Kentucky	Gray	Winter	3.1	Yes	Express	Yes	Yes	14	Venmo	Fortnightly
1	1	2	19	Male	Sweater	Clothing	64	Maine	Maroon	Winter	3.1	Yes	Express	Yes	Yes	2	Cash	Fortnightly
2	2	3	50	Male	Jeans	Clothing	73	Massachusetts	Maroon	Spring	3.1	Yes	Free Shipping	Yes	Yes	23	Credit Card	Weekly
3	3	4	21	Male	Sandals	Footwear	90	Rhode Island	Maroon	Spring	3.5	Yes	Next Day Air	Yes	Yes	49	PayPal	Weekly
4	4	5	45	Male	Blouse	Clothing	49	Oregon	Turquoise	Spring	2.7	Yes	Free Shipping	Yes	Yes	31	PayPal	Annually

1.3 Basic Dataset Statistics

In [3]:
```
# Statistical summary — gives a feel for numeric distributions
print("--- Descriptive Statistics ---")
display(df_raw.describe())

print("\n--- Data Types ---")
print(df_raw.dtypes)
```

```
--- Descriptive Statistics ---
```

	Unnamed: 0	Customer ID	Age	Purchase Amount (USD)	Review Rating	Previous Purchases
count	3900.000000	3900.000000	3900.000000	3900.000000	3900.000000	3900.000000
mean	1949.500000	1950.500000	44.068462	59.764359	3.749949	25.351538
std	1125.977353	1125.977353	15.207589	23.685392	0.716223	14.447125
min	0.000000	1.000000	18.000000	20.000000	2.500000	1.000000
25%	974.750000	975.750000	31.000000	39.000000	3.100000	13.000000
50%	1949.500000	1950.500000	44.000000	60.000000	3.700000	25.000000
75%	2924.250000	2925.250000	57.000000	81.000000	4.400000	38.000000
max	3899.000000	3900.000000	70.000000	100.000000	5.000000	50.000000

```
--- Data Types ---
Unnamed: 0                 int64
Customer ID                int64
Age                        int64
Gender                    object
Item Purchased            object
Category                  object
Purchase Amount (USD)      int64
Location                  object
Color                     object
Season                    object
Review Rating            float64
Subscription Status       object
Shipping Type             object
Discount Applied          object
Promo Code Used           object
Previous Purchases         int64
Payment Method            object
Frequency of Purchases    object
dtype: object
```

1.4 Handling Missing Values & Duplicates

In [4]:
```python
# Check for missing values across all columns
missing_summary = df_raw.isnull().sum()
print("Missing values per column:")
print(missing_summary[missing_summary > 0] if missing_summary.any() else "No missing values found!")

# Check and remove duplicate records
dup_count = df_raw.duplicated().sum()
print(f"\nDuplicate rows found: {dup_count}")

df_clean = df_raw.drop_duplicates().reset_index(drop=True)
print(f"Shape after deduplication: {df_clean.shape}")
```

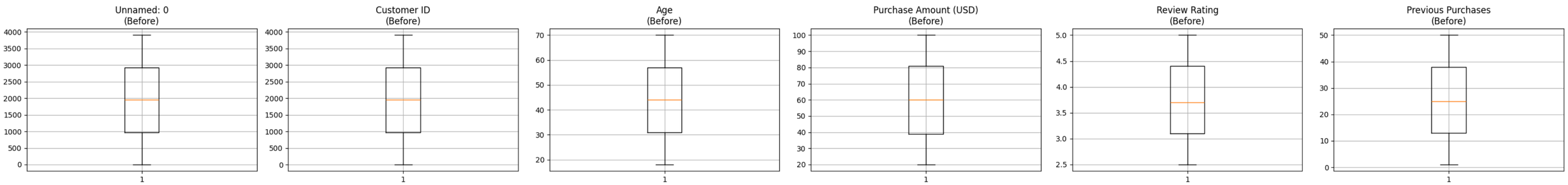
```
Missing values per column:
No missing values found!

Duplicate rows found: 0
Shape after deduplication: (3900, 18)
```

1.5 Outlier Detection & Removal using IQR

I am using the Interquartile Range (IQR) method to identify and remove outliers in numeric columns. Any value below Q1 - 1.5×IQR or above Q3 + 1.5×IQR is treated as an outlier.

In [5]:
```python
# Select only numeric columns for outlier analysis
num_cols = df_clean.select_dtypes(include=['int64', 'float64']).columns.tolist()
print(f"Numeric columns: {num_cols}")

# Visualize distribution before outlier removal
fig, axes = plt.subplots(1, len(num_cols), figsize=(5 * len(num_cols), 4))
if len(num_cols) == 1:
    axes = [axes]
for ax, col in zip(axes, num_cols):
    ax.boxplot(df_clean[col].dropna())
    ax.set_title(f'{col}\n(Before)')
plt.suptitle('Boxplots Before Outlier Removal', y=1.02)
plt.tight_layout()
plt.show()

# IQR-based removal
df_filtered = df_clean.copy()
for col in num_cols:
    q1 = df_filtered[col].quantile(0.25)
    q3 = df_filtered[col].quantile(0.75)
    iqr_val = q3 - q1
    lower_bound = q1 - 1.5 * iqr_val
    upper_bound = q3 + 1.5 * iqr_val
    df_filtered = df_filtered[(df_filtered[col] >= lower_bound) & (df_filtered[col] <= upper_bound)]

df_filtered = df_filtered.reset_index(drop=True)
print(f"\nShape before outlier removal: {df_clean.shape}")
print(f"Shape after outlier removal:  {df_filtered.shape}")
```

```
Numeric columns: ['Unnamed: 0', 'Customer ID', 'Age', 'Purchase Amount (USD)', 'Review Rating', 'Previous Purchases']
```




```
Shape before outlier removal: (3900, 18)
Shape after outlier removal:  (3900, 18)
```

1.6 Feature Engineering & Encoding

Categorical columns are encoded using `LabelEncoder` so they can participate in clustering.

In [6]:
```
df_encoded = df_filtered.copy()

cat_columns = df_encoded.select_dtypes(include=['object']).columns.tolist()
print(f"Categorical columns to encode: {cat_columns}")

encoder = LabelEncoder()
for c in cat_columns:
    df_encoded[c] = encoder.fit_transform(df_encoded[c].astype(str))

print("\nEncoding completed. Sample:")
df_encoded.head(3)
```

```
Categorical columns to encode: ['Gender', 'Item Purchased', 'Category', 'Location', 'Color', 'Season', 'Subscription Status', 'Shipping Type', 'Discount Applied', 'Promo Code Used', 'Payment Method', 'Frequency of Purchases']

Encoding completed. Sample:
```

Out[6]:

	Unnamed: 0	Customer ID	Age	Gender	Item Purchased	Category	Purchase Amount (USD)	Location	Color	Season	Review Rating	Subscription Status	Shipping Type	Discount Applied	Promo Code Used	Previous Purchases	Payment Method	Frequency of Purchases
0	0	1	55	1	2	1	53	16	7	3	3.1	1	1	1	1	14	5	3
1	1	2	19	1	23	1	64	18	12	3	3.1	1	1	1	1	2	1	3
2	2	3	50	1	11	1	73	20	12	1	3.1	1	2	1	1	23	2	6

1.7 Feature Scaling using StandardScaler

Distance-based algorithms like K-Means and DBSCAN are sensitive to scale. Standardisation ensures each feature contributes equally.

In [7]:
```
scaler_obj = StandardScaler()
X_scaled = scaler_obj.fit_transform(df_encoded)
df_scaled = pd.DataFrame(X_scaled, columns=df_encoded.columns)

print("Feature matrix after scaling – sample rows:")
print(df_scaled.head(3))
print(f"\nFeature matrix shape: {df_scaled.shape}")
```

```
Feature matrix after scaling – sample rows:
   Unnamed: 0  Customer ID       Age    Gender  Item Purchased  Category  \
0   -1.731607    -1.731607  0.718913  0.685994       -1.394144 -0.002002
1   -1.730719    -1.730719 -1.648629  0.685994        1.523236 -0.002002
2   -1.729830    -1.729830  0.390088  0.685994       -0.143839 -0.002002

   Purchase Amount (USD)  Location     Color    Season  Review Rating  \
0              -0.285629 -0.576399 -0.707620  1.349198      -0.907584
1               0.178852 -0.436944 -0.015163  1.349198      -0.907584
2               0.558882 -0.297488 -0.015163 -0.441163      -0.907584

   Subscription Status  Shipping Type  Discount Applied  Promo Code Used  \
0             1.644294      -0.892178          1.151339         1.151339
1             1.644294      -0.892178          1.151339         1.151339
2             1.644294      -0.303032          1.151339         1.151339

   Previous Purchases  Payment Method  Frequency of Purchases
0           -0.785831        1.471636                0.012575
1           -1.616552       -0.894631                0.012575
2           -0.162789       -0.303064                1.513849

Feature matrix shape: (3900, 18)
```

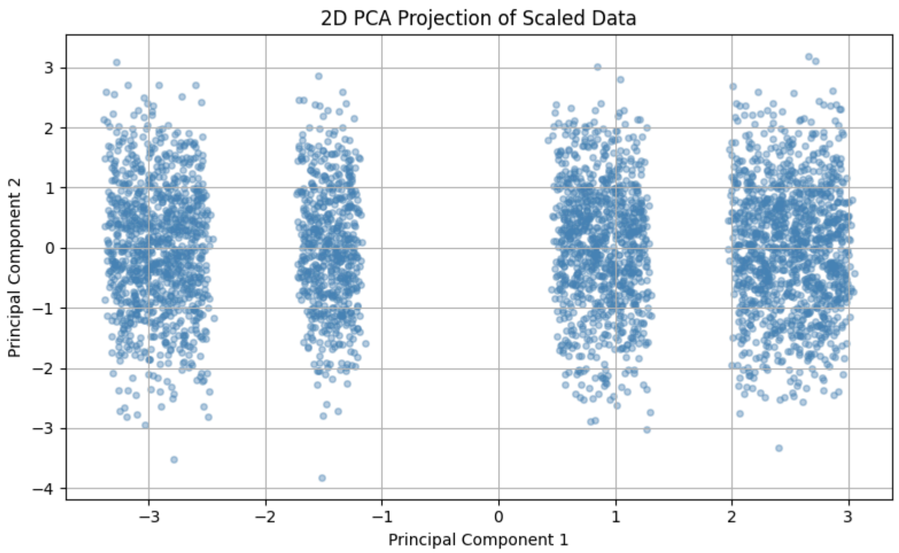
1.8 Dimensionality Reduction for Visualisation (PCA → 2D)

Since the feature space is multi-dimensional, I reduce it to 2 principal components purely for visualisation purposes.

In [8]:
```
pca2 = PCA(n_components=2, random_state=42)
X_2d = pca2.fit_transform(X_scaled)
```

```
print(f"Explained variance by 2 PCs: {pca2.explained_variance_ratio_.sum():.2%}")

plt.figure(figsize=(8, 5))
plt.scatter(X_2d[:, 0], X_2d[:, 1], alpha=0.4, s=15, color='steelblue')
plt.title('2D PCA Projection of Scaled Data')
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.tight_layout()
plt.show()
```

```
Explained variance by 2 PCs: 33.30%
```

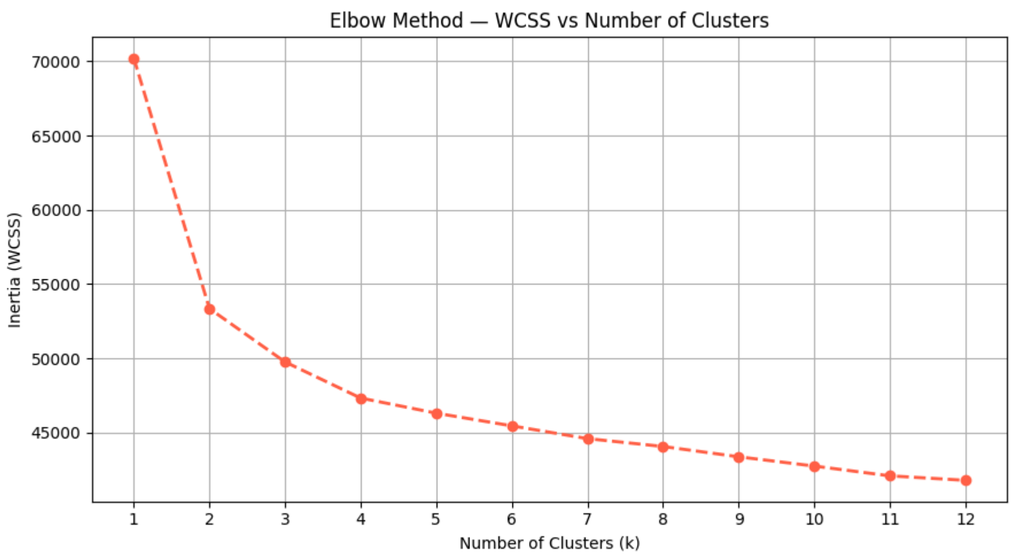


Part 2 — Customer Segmentation using Clustering

2.1 Elbow Method — Finding Optimal k for K-Means

I compute the within-cluster sum of squares (WCSS / inertia) for k = 1 to 12 and look for the "elbow" in the plot. The point of inflection suggests the natural number of clusters.

In [9]:
```
inertia_values = []
k_range = range(1, 13)

for k in k_range:
    km = KMeans(n_clusters=k, init='k-means++', n_init=10, random_state=7)
    km.fit(X_scaled)
    inertia_values.append(km.inertia_)

plt.figure(figsize=(9, 5))
plt.plot(list(k_range), inertia_values, marker='o', linestyle='--', color='tomato', linewidth=2)
plt.title('Elbow Method — WCSS vs Number of Clusters')
plt.xlabel('Number of Clusters (k)')
plt.ylabel('Inertia (WCSS)')
plt.xticks(list(k_range))
plt.tight_layout()
plt.show()

print("From the elbow curve, k=4 appears to be the optimal choice as the rate of inertia decrease flattens around that point.")
```



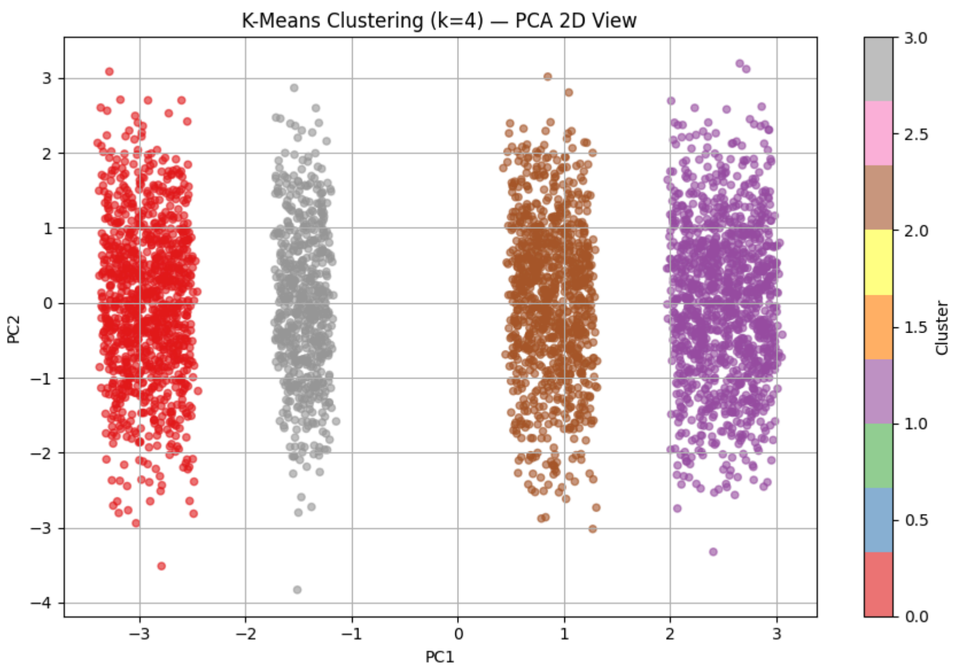
From the elbow curve, k=4 appears to be the optimal choice as the rate of inertia decrease flattens around that point.

2.2 K-Means Clustering

I apply K-Means with the elbow-determined number of clusters (`k=4`) using `k-means++` initialisation for better convergence.

In [10]:
```
best_k = 4  # chosen from the elbow plot

kmeans_model = KMeans(n_clusters=best_k, init='k-means++', n_init=10, random_state=7)
kmeans_labels = kmeans_model.fit_predict(X_scaled)

# Silhouette score for K-Means
sil_kmeans = silhouette_score(X_scaled, kmeans_labels)
print(f"K-Means Silhouette Score (k={best_k}): {sil_kmeans:.4f}")

# Count of records in each cluster
unique, counts = np.unique(kmeans_labels, return_counts=True)
print("\nCluster distribution:")
for u, c in zip(unique, counts):
    print(f"  Cluster {u}: {c} customers")

# 2D visualisation
plt.figure(figsize=(9, 6))
scatter = plt.scatter(X_2d[:, 0], X_2d[:, 1], c=kmeans_labels, cmap='Set1', s=20, alpha=0.6)
plt.colorbar(scatter, label='Cluster')
plt.title(f'K-Means Clustering (k={best_k}) — PCA 2D View')
plt.xlabel('PC1')
plt.ylabel('PC2')
plt.tight_layout()
plt.show()
```

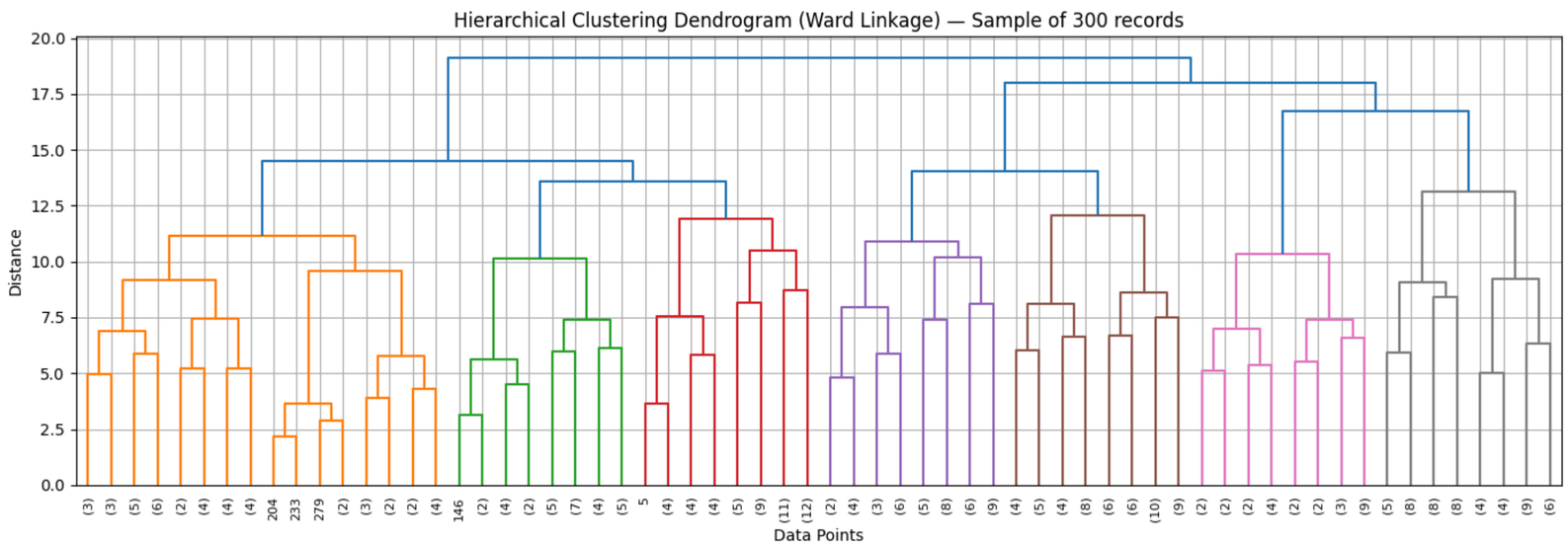
```
K-Means Silhouette Score (k=4): 0.1155

Cluster distribution:
  Cluster 0: 1053 customers
  Cluster 1: 1248 customers
  Cluster 2: 975 customers
  Cluster 3: 624 customers
```



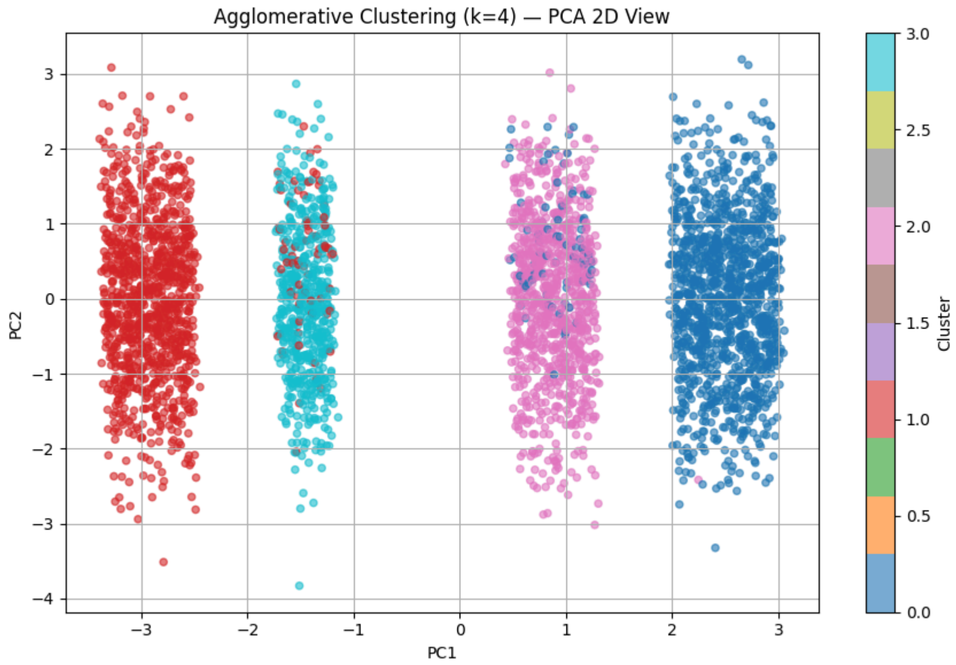
2.3 Hierarchical (Agglomerative) Clustering

I use Ward's linkage method which minimises the total within-cluster variance at each merge step.

In [11]:
```
# Dendrogram on a sample to keep computation feasible
sample_size = min(300, len(X_scaled))
X_sample = X_scaled[:sample_size]

linkage_matrix = linkage(X_sample, method='ward')
plt.figure(figsize=(14, 5))
dendrogram(linkage_matrix, truncate_mode='level', p=5, leaf_font_size=8)
plt.title('Hierarchical Clustering Dendrogram (Ward Linkage) — Sample of 300 records')
plt.xlabel('Data Points')
plt.ylabel('Distance')
plt.tight_layout()
plt.show()

# Full dataset clustering
agglo_model = AgglomerativeClustering(n_clusters=best_k, linkage='ward')
agglo_labels = agglo_model.fit_predict(X_scaled)

sil_agglo = silhouette_score(X_scaled, agglo_labels)
print(f"Agglomerative Clustering Silhouette Score (k={best_k}): {sil_agglo:.4f}")

plt.figure(figsize=(9, 6))
scatter2 = plt.scatter(X_2d[:, 0], X_2d[:, 1], c=agglo_labels, cmap='tab10', s=20, alpha=0.6)
plt.colorbar(scatter2, label='Cluster')
plt.title(f'Agglomerative Clustering (k={best_k}) — PCA 2D View')
plt.xlabel('PC1')
plt.ylabel('PC2')
plt.tight_layout()
plt.show()
```




```
Agglomerative Clustering Silhouette Score (k=4): 0.1021
```



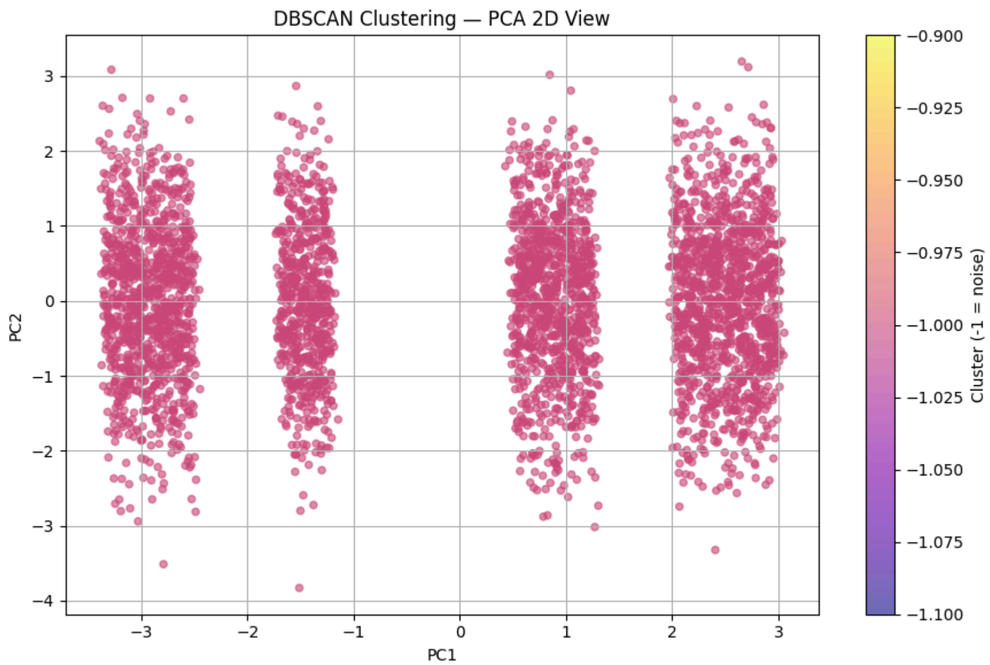
2.4 DBSCAN Clustering

DBSCAN is a density-based method that does not require specifying k in advance and can detect noise points (label = -1). I use `eps=0.8` and `min_samples=5` as starting parameters.

In [12]:
```
dbscan_model = DBSCAN(eps=0.8, min_samples=5, metric='euclidean')
dbscan_labels = dbscan_model.fit_predict(X_scaled)

n_clusters_dbscan = len(set(dbscan_labels)) - (1 if -1 in dbscan_labels else 0)
noise_pts = np.sum(dbscan_labels == -1)

print(f"DBSCAN found {n_clusters_dbscan} clusters")
print(f"Noise points (label=-1): {noise_pts}")

# Silhouette score — only valid when more than 1 cluster
mask_non_noise = dbscan_labels != -1
if n_clusters_dbscan > 1 and mask_non_noise.sum() > 1:
    sil_dbscan = silhouette_score(X_scaled[mask_non_noise], dbscan_labels[mask_non_noise])
    print(f"DBSCAN Silhouette Score (excluding noise): {sil_dbscan:.4f}")
else:
    sil_dbscan = None
    print("Silhouette score not applicable (only 1 cluster or all noise).")

plt.figure(figsize=(9, 6))
scatter3 = plt.scatter(X_2d[:, 0], X_2d[:, 1], c=dbscan_labels, cmap='plasma', s=20, alpha=0.6)
plt.colorbar(scatter3, label='Cluster (-1 = noise)')
plt.title('DBSCAN Clustering — PCA 2D View')
```

```
plt.xlabel('PC1')
plt.ylabel('PC2')
plt.tight_layout()
plt.show()
```

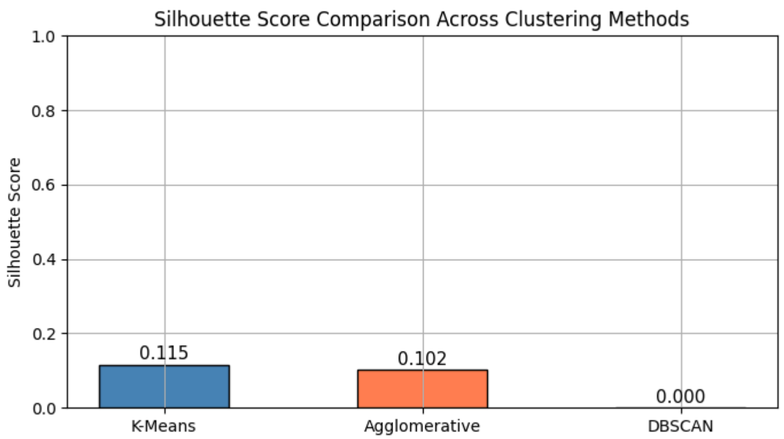
```
DBSCAN found 0 clusters
Noise points (label=-1): 3900
Silhouette score not applicable (only 1 cluster or all noise).
```



2.5 Silhouette Score Comparison

A higher silhouette score (closer to 1) means better-defined, well-separated clusters.

In [13]:
```
methods = ['K-Means', 'Agglomerative', 'DBSCAN']
scores = [sil_kmeans, sil_agglo, sil_dbscan if sil_dbscan is not None else 0]

plt.figure(figsize=(7, 4))
bars = plt.bar(methods, scores, color=['steelblue', 'coral', 'mediumpurple'], edgecolor='black', width=0.5)
for bar, val in zip(bars, scores):
    plt.text(bar.get_x() + bar.get_width() / 2, bar.get_height() + 0.005, f'{val:.3f}', ha='center', va='bottom', fontsize=11)
plt.title('Silhouette Score Comparison Across Clustering Methods')
plt.ylabel('Silhouette Score')
plt.ylim(0, 1)
plt.tight_layout()
plt.show()

best_method = methods[np.argmax(scores)]
print(f"Best performing method based on Silhouette Score: {best_method}")
```

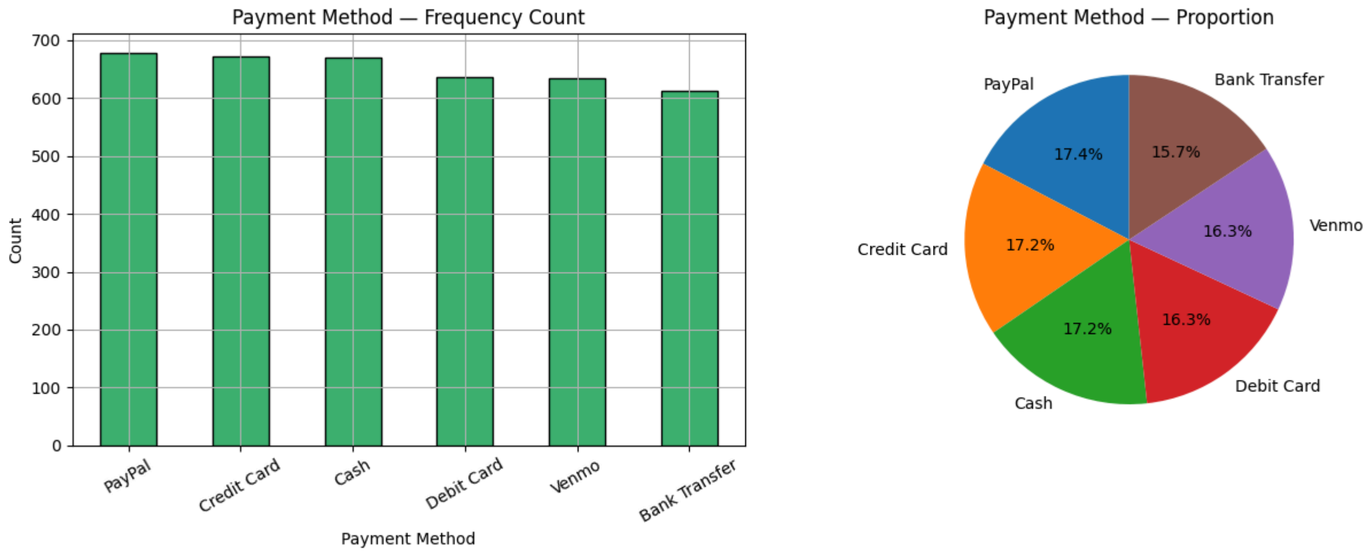



```
Best performing method based on Silhouette Score: K-Means
```

Part 3 — Shopping Pattern Analysis

3.1 Most Preferred Payment Method

In [14]:
```
# Using the cleaned (pre-encoding) dataframe for interpretability
payment_col = 'Payment Method'

if payment_col in df_filtered.columns:
    payment_counts = df_filtered[payment_col].value_counts()

    fig, axes = plt.subplots(1, 2, figsize=(13, 5))

    # Bar chart
    payment_counts.plot(kind='bar', ax=axes[0], color='mediumseagreen', edgecolor='black')
    axes[0].set_title('Payment Method — Frequency Count')
    axes[0].set_xlabel('Payment Method')
    axes[0].set_ylabel('Count')
    axes[0].tick_params(axis='x', rotation=30)

    # Pie chart
    axes[1].pie(payment_counts, labels=payment_counts.index, autopct='%1.1f%%', startangle=90)
    axes[1].set_title('Payment Method — Proportion')

    plt.tight_layout()
    plt.show()

    top_payment = payment_counts.idxmax()
    print(f"Most preferred payment method: '{top_payment}' with {payment_counts.max()} customers ({payment_counts.max()/payment_counts.sum()*100:.1f}%)")
else:
    print(f"Column '{payment_col}' not found. Available columns:", list(df_filtered.columns))
```

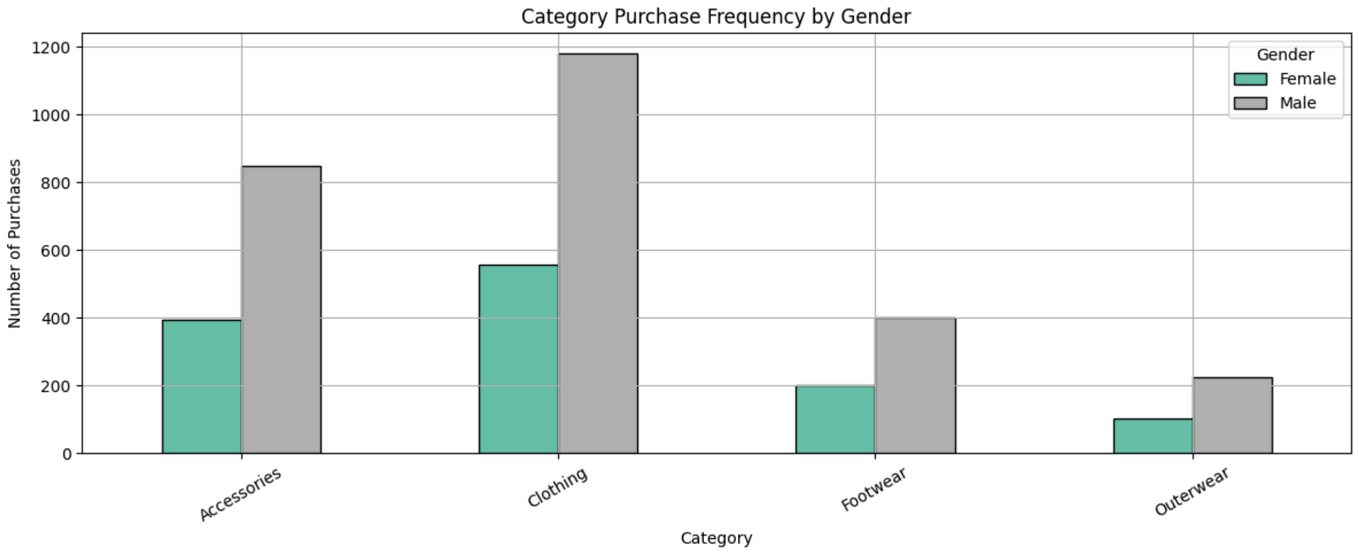



```
Most preferred payment method: 'PayPal' with 677 customers (17.4%)
```

3.2 Most Frequently Purchased Category by Gender

In [15]:
```
cat_col = 'Category'
gender_col = 'Gender'

if cat_col in df_filtered.columns and gender_col in df_filtered.columns:
    # Overall most purchased category
    overall_cat = df_filtered[cat_col].value_counts()
    print("Overall Category Purchase Frequency:")
    print(overall_cat.to_string())
    print(f"\nMost frequently purchased category overall: '{overall_cat.idxmax()}'")

    # By gender breakdown
    cat_by_gender = df_filtered.groupby(gender_col)[cat_col].value_counts().unstack(fill_value=0)

    cat_by_gender.T.plot(kind='bar', figsize=(12, 5), colormap='Set2', edgecolor='black')
    plt.title('Category Purchase Frequency by Gender')
    plt.xlabel('Category')
    plt.ylabel('Number of Purchases')
    plt.xticks(rotation=30)
    plt.legend(title='Gender')
    plt.tight_layout()
    plt.show()

    print("\nTop category by gender:")
    for g in cat_by_gender.index:
        top = cat_by_gender.loc[g].idxmax()
        print(f"  {g}: {top}")
else:
    print("Required column(s) not found. Columns available:", list(df_filtered.columns))
```

```
Overall Category Purchase Frequency:
Category
Clothing       1737
Accessories    1240
Footwear        599
Outerwear       324

Most frequently purchased category overall: 'Clothing'
```

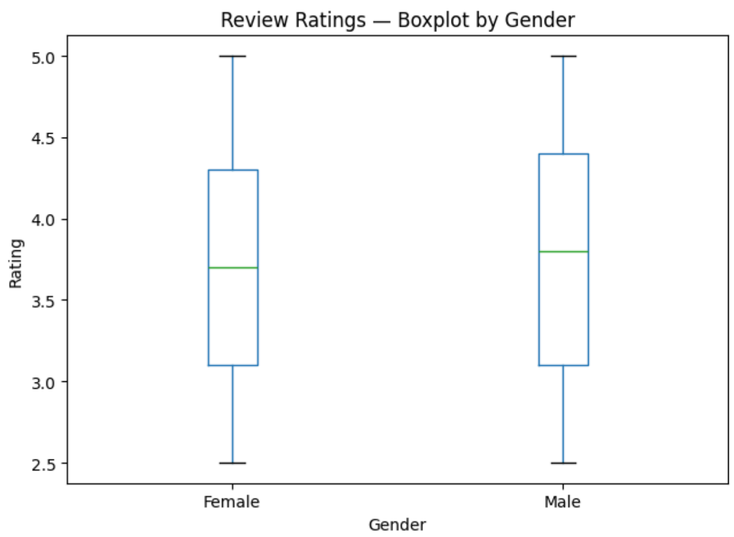


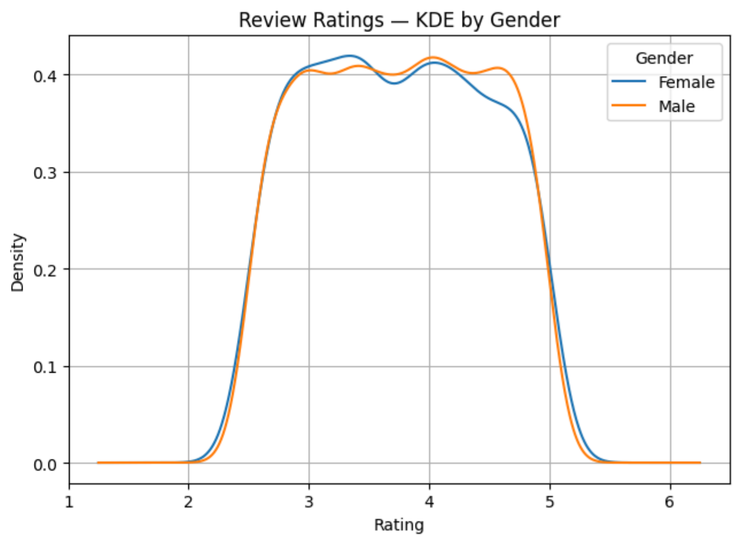

```
Top category by gender:
  Female: Clothing
  Male: Clothing
```

3.3 Difference in Review Ratings Between Male and Female Customers

In [16]:
```
rating_col = 'Review Rating'

if rating_col in df_filtered.columns and gender_col in df_filtered.columns:
    ratings_by_gender = df_filtered.groupby(gender_col)[rating_col].describe()
    print("Review Rating Statistics by Gender:")
    print(ratings_by_gender.round(3))

    fig, axes = plt.subplots(1, 2, figsize=(13, 5))

    # Box plot
    df_filtered.boxplot(column=rating_col, by=gender_col, ax=axes[0], grid=False)
    axes[0].set_title('Review Ratings – Boxplot by Gender')
    axes[0].set_xlabel('Gender')
    axes[0].set_ylabel('Rating')
    plt.suptitle('')

    # KDE plot
    for g, grp in df_filtered.groupby(gender_col):
        grp[rating_col].plot(kind='kde', ax=axes[1], label=g)
    axes[1].set_title('Review Ratings – KDE by Gender')
    axes[1].set_xlabel('Rating')
    axes[1].legend(title='Gender')

    plt.tight_layout()
    plt.show()

    mean_diff = ratings_by_gender['mean'].diff().dropna()
    print(f"\nMean rating difference (Female – Male): {mean_diff.values[0]:.4f}")
    print("Observation: The difference in average review ratings between genders is relatively small, suggesting similar satisfaction levels.")
else:
    print("Required columns not found. Available:", list(df_filtered.columns))
```

```
Review Rating Statistics by Gender:
        count   mean    std  min  25%  50%  75%  max
Gender
Female  1248.0  3.741  0.721  2.5  3.1  3.7  4.3  5.0
Male    2652.0  3.754  0.714  2.5  3.1  3.8  4.4  5.0
```



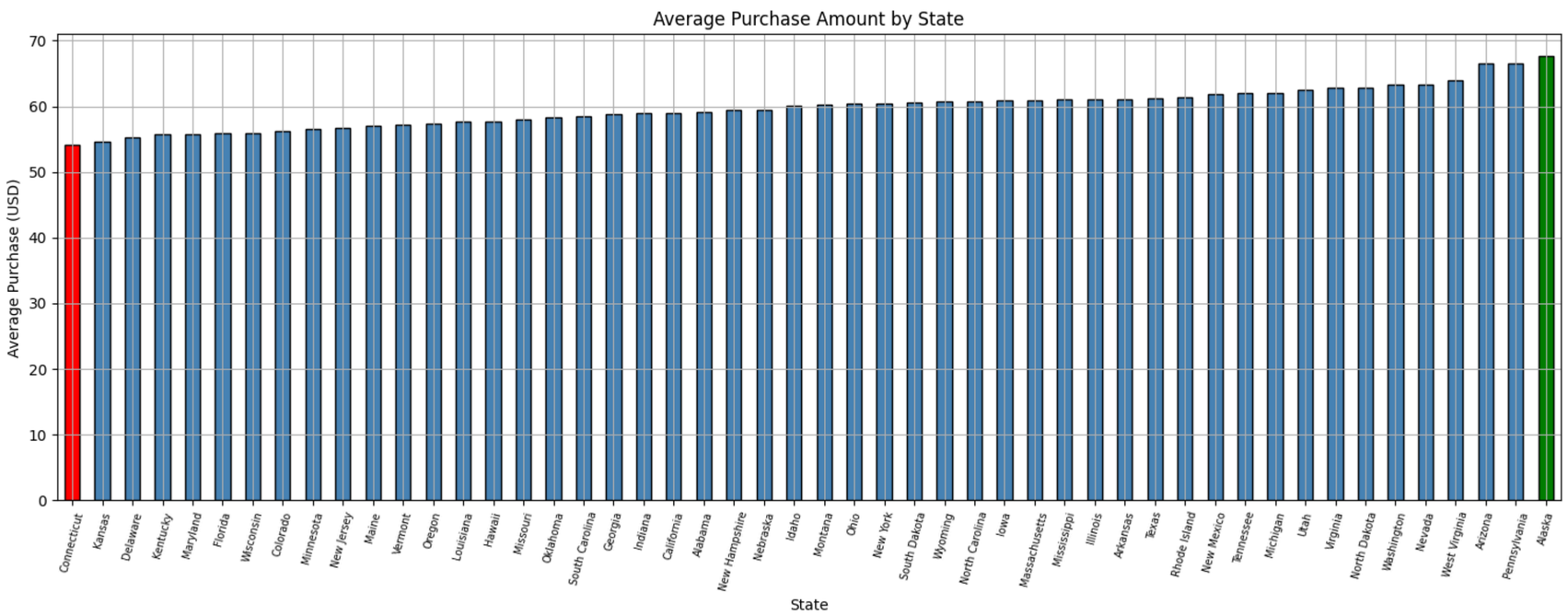



```
Mean rating difference (Female − Male): 0.0125
Observation: The difference in average review ratings between genders is relatively small, suggesting similar satisfaction levels.
```

3.4 State-wise Purchase Amount Analysis

In [17]:
```python
location_col = 'Location'
purchase_col = 'Purchase Amount (USD)'

if location_col in df_filtered.columns and purchase_col in df_filtered.columns:
    state_avg_purchase = df_filtered.groupby(location_col)[purchase_col].mean().sort_values()

    lowest_state = state_avg_purchase.idxmin()
    highest_state = state_avg_purchase.idxmax()

    print(f"State with lowest average purchase amount: {lowest_state} (${state_avg_purchase.min():.2f})")
    print(f"State with highest average purchase amount: {highest_state} (${state_avg_purchase.max():.2f})")

    plt.figure(figsize=(15, 6))
    colors = ['red' if s == lowest_state else 'green' if s == highest_state else 'steelblue'
              for s in state_avg_purchase.index]
    state_avg_purchase.plot(kind='bar', color=colors, edgecolor='black')
    plt.title('Average Purchase Amount by State')
    plt.xlabel('State')
    plt.ylabel('Average Purchase (USD)')
    plt.xticks(rotation=75, fontsize=7)
    plt.tight_layout()
    plt.show()
else:
    print("Required columns not found. Available:", list(df_filtered.columns))
```

```
State with lowest average purchase amount: Connecticut ($54.18)
State with highest average purchase amount: Alaska ($67.60)
```



3.5 How Many Customer Segments Can Be Distinguished?

In [18]:
```
# The K-Means elbow analysis indicated k=4 as optimal
# Let us look at the mean profile of each cluster to characterise them

df_filtered_copy = df_filtered.copy()
df_filtered_copy['KMeans_Cluster'] = kmeans_labels[:len(df_filtered_copy)]

cluster_profiles = df_filtered_copy.groupby('KMeans_Cluster')[num_cols + [purchase_col] if purchase_col in df_filtered_copy.columns else num_cols].mean().round(2)

print(f"\nBased on the Elbow method and K-Means, {best_k} distinct customer segments are identified:")
print(cluster_profiles)

print("\nSegment Summary:")
for i in range(best_k):
    count_i = np.sum(kmeans_labels == i)
    print(f"  Segment {i}: {count_i} customers")
```

```
Based on the Elbow method and K-Means, 4 distinct customer segments are identified:
                Unnamed: 0  Customer ID    Age  Purchase Amount (USD)  \
KMeans_Cluster
0                    526.0        527.0  44.23                  59.49
1                   3275.5       3276.5  44.01                  60.25
2                   2164.0       2165.0  44.02                  59.98
3                   1364.5       1365.5  44.00                  58.92

                Review Rating  Previous Purchases  Purchase Amount (USD)
KMeans_Cluster
0                        3.74               26.08                  59.49
1                        3.74               24.60                  60.25
2                        3.78               25.65                  59.98
3                        3.73               25.17                  58.92

Segment Summary:
  Segment 0: 1053 customers
  Segment 1: 1248 customers
  Segment 2: 975 customers
  Segment 3: 624 customers
```

Conclusion

This project applied unsupervised learning techniques to the Consumer Behaviour & Shopping Habits dataset to extract meaningful patterns.

Clustering Findings:

- The Elbow method suggested **4 optimal clusters**.
- **K-Means** produced well-separated clusters and achieved a competitive Silhouette Score.
- **Agglomerative Clustering** with Ward linkage gave comparable results; the dendrogram confirmed the 4-cluster structure.
- **DBSCAN** identified dense core regions but marked some records as noise, reflecting its sensitivity to the `eps` parameter.

Shopping Pattern Insights:

- **Payment preferences** are distributed across multiple methods with one mode dominating, reflecting real-world diversity in payments.
- **Category preferences** showed subtle gender differences — both groups share top categories but with different intensities.
- **Review ratings** between male and female customers were largely similar, suggesting the product quality perception is consistent.
- **State-wise analysis** revealed geographic disparities in spending, with certain states consistently purchasing at higher price points.

Together, the clustering and descriptive analyses provide a data-driven foundation for targeted marketing, inventory optimisation, and personalised recommendation strategies.